

Generación Multimodal e Iterativa con Gemini

¿Qué es la Multimodalidad Nativa?

A diferencia de otros sistemas que usan un modelo para el texto (como GPT-4) y otro distinto para la imagen (como DALL-E), **Gemini 2.x (Nano Banana)** es un modelo **nativo multimodal**.

- **Significado:** El modelo procesa y genera diferentes tipos de información (texto, código, imágenes) en el mismo espacio neuronal.
- **Ventaja:** Hay una coherencia semántica mucho mayor. Si el modelo escribe un cuento sobre un "caballero con armadura de cristal", la imagen reflejará exactamente ese concepto porque la "idea" es la misma en ambas modalidades.

El Concepto de Iteración en IA Generativa

La IA rara vez genera el resultado perfecto al primer intento. La **iteración** es el proceso de:

1. **Generación inicial:** Crear una base a partir de un prompt.
2. **Evaluación:** El usuario identifica qué falta o qué sobra.
3. **Refinamiento (Feedback loop):** Se le da una instrucción correctiva al modelo manteniendo el contexto anterior. En esta práctica, simularemos esto combinando el prompt original con las nuevas instrucciones del usuario.

Enfoque Gemini 2.x (Nativo): El modelo genera el texto y los píxeles de forma **simultánea y entrelazada**. El cerebro de Gemini "ve" la imagen que está creando mientras escribe el texto, lo que garantiza que lo que describe el texto y lo que muestra la imagen coincidan casi a la perfección.

El parámetro `response_modalities=["TEXT", "IMAGE"]`

¿Qué es técnicamente?

Es una instrucción de configuración que le indica al modelo qué "lenguajes" de salida tiene permiso para utilizar.

- Por defecto, casi todos los modelos de IA están configurados solo como ["TEXT"].
- Al añadir "IMAGE", estamos activando el motor **Nano Banana** dentro del mismo flujo de pensamiento del modelo. No es una llamada a una herramienta externa, es el modelo decidiendo generar píxeles en lugar de palabras.

¿Cómo funciona el flujo de respuesta?

Cuando activas ambas modalidades, la respuesta que devuelve el servidor no es un simple String (cadena de texto). Es un objeto compuesto por una lista de **Partes** (parts).

Si usas `response_modalities=["TEXT", "IMAGE"]`, el modelo puede devolver:

1. `part.text`: Un fragmento de texto.
2. `part.inline_data`: Un bloque de bytes que representan la imagen (generalmente en formato PNG o JPEG).

Práctica:

Título: Storytelling Visual e Iteración con Gemini 2.5

Objetivo: Crear un sistema que genere una historia ilustrada y permita al usuario "dirigir" al artista digital para mejorar el resultado.

Tarea:

Escribe un script en Python que siga este flujo de trabajo:

1. **Paso 1 (Generación):** Solicita al usuario un "Personaje" y un "Escenario". Envía un prompt a Gemini para que genere una leyenda breve y una imagen cinematográfica.
2. **Paso 2 (Visualización):** Muestra el texto por consola y abre la imagen generada.
3. **Paso 3 (Feedback):** Pregunta al usuario: *"¿Qué cambios te gustaría hacer?"*.
4. **Paso 4 (Iteración):** Realiza una segunda llamada al modelo enviando: **Prompt Original + Feedback del usuario**. Guarda y muestra la nueva imagen como `portada_final.png`.

Requisitos:

1. Utiliza el nuevo SDK google-genai.
2. Instalación de dependencias: Ejecutar:
`pip install -U google-genai Pillow`
3. Modelo gemini-2.5-flash-image, si diera error (por estar en fase experimental), usar gemini-2.0-flash.

```
from google import genai
from google.genai import types
from PIL import Image
import io
# Configuración del cliente (Asegúrate de tener la API KEY en tu entorno)
client = genai.Client(api_key="TU_API_KEY")
```

4. Utiliza `response_modalities`

```
response = client.models.generate_content(
    model="gemini-2.0-flash", # 0 gemini-2.5-flash-image
    contents=prompt_texto,
    config=types.GenerateContentConfig(
        response_modalities=["TEXT", "IMAGE"] )
```

5. Utiliza el historial del modelo o en cada iteración almacena el prompt.
6. Investiga `as_image()` : La librería google-genai incluye métodos de para transformar los bytes de `inline_data` directamente en un objeto de la librería `Pillow` para tratar las imágenes.